Design and Implementation of a Multi-Layer Perceptron (MLP) for Handwritten Digit Recognition

Taham Rahim (25I-3044)

March 28, 2026

Abstract

This report details the end-to-end development of a handwritten digit recognition system using a 3-layer Neural Network built from scratch using NumPy. The implementation covers data preprocessing of the MNIST dataset, forward propagation through dense layers, and backpropagation using the chain rule. By integrating a Momentum-based optimizer and Sigmoid activation functions, the model achieved a final test accuracy of **95.54%**.

1 Introduction

Handwritten digit recognition is a fundamental problem in computer vision. The MNIST dataset, consisting of 70,000 28×28 grayscale images, serves as the benchmark for this task. In this project, we implement a Feed-Forward Neural Network (FFNN) without the use of deep learning frameworks like TensorFlow or PyTorch. The objective is to demonstrate the efficacy of manual gradient derivation and stochastic gradient descent in solving high-dimensional classification problems.

2 Mathematical Methodology

2.1 Forward Propagation

Each layer L in the network performs an affine transformation followed by a non-linear activation. Given input $A^{(L-1)}$, the pre-activation $Z^{(L)}$ is computed as:

$$Z^{(L)} = A^{(L-1)}W^{(L)} + b^{(L)} \tag{1}$$

The activation $A^{(L)}$ is then computed using the Sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{2}$$

2.2 Backpropagation and Gradient Descent

To update the weights and minimize the Mean Squared Error (MSE), we calculate the gradient of the loss function C with respect to each parameter. For the output layer, the error term $\delta^{(3)}$ is defined as:

$$\delta^{(3)} = \nabla_A C \odot \sigma'(Z^{(3)}) = (A^{(3)} - Y) \odot (A^{(3)}(1 - A^{(3)})) \tag{3}$$

Using the chain rule, we propagate this error backward to update $W^{(1)}, W^{(2)}$, and $W^{(3)}$.

3 Task 7: Implementation Details

3.1 Activation Function and Numerical Stability

The Sigmoid function is prone to overflow during exponential calculations. To ensure numerical stability, we utilize `np.clip` to restrict the range of the input z.

```
def sigmoid(self, z):
    # Clips values to avoid overflow error in exp calculation
    return 1.0 / (1.0 + np.exp(-np.clip(z, -500, 500)))

def sigmoid_derivative(self, a):
    # f'(z) = a(1-a) where a = sigmoid(z)
    return a * (1.0 - a)
```

3.2 Weight Initialization and Biases

Weights are initialized using a uniform distribution between -0.1 and 0.1 to break symmetry. The **Biases** are initialized to zero, allowing the network to shift the activation function independently of the weights.

```
# Task 1: Initialization
np.random.seed(42)
self.W1 = np.random.uniform(-0.1, 0.1, (784, 128))
self.b1 = np.zeros((1, 128))
self.W2 = np.random.uniform(-0.1, 0.1, (128, 64))
self.b2 = np.zeros((1, 64))
```

3.3 The Momentum Optimizer

To accelerate convergence and dampen oscillations in the gradient, we implement Momentum ($\beta = 0.9$). This allows the updates to build "velocity" in directions of consistent gradients.

```
def update_parameters(self, grads):
    dW1, db1, dW2, db2, dW3, db3 = grads
    # Update velocities for Momentum
    self.vW1 = self.beta * self.vW1 + (1 - self.beta) * dW1
    self.vb1 = self.beta * self.vb1 + (1 - self.beta) * db1
    # Apply Updates
    self.W1 -= self.lr * self.vW1
    self.b1 -= self.lr * self.vb1
```

4 Data Pipeline and Training

4.1 Task 5: Preprocessing and Shuffling

The MNIST data is normalized by dividing pixel values by 255.0 to bring them into the $[0, 1]$ range. During training, the data is shuffled every epoch to ensure the network does not learn the order of samples, which prevents overfitting.

```
for epoch in range(10):
    # Randomly shuffle data indices
    idx = np.random.permutation(X_train.shape[0])
    X_shuf, Y_shuf = X_train[idx], Y_train_oh[idx]

    for i in range(0, X_train.shape[0], 64):
        X_batch = X_shuf[i : i + 64]
        Y_batch = Y_shuf[i : i + 64]
        # Forward -> Backward -> Update
```

5 Results and Discussion

5.1 Training Convergence

As shown in Figure 1, the MSE loss exhibits a sharp decline in the first three epochs, eventually plateauing as the network converges on an optimal weight configuration.

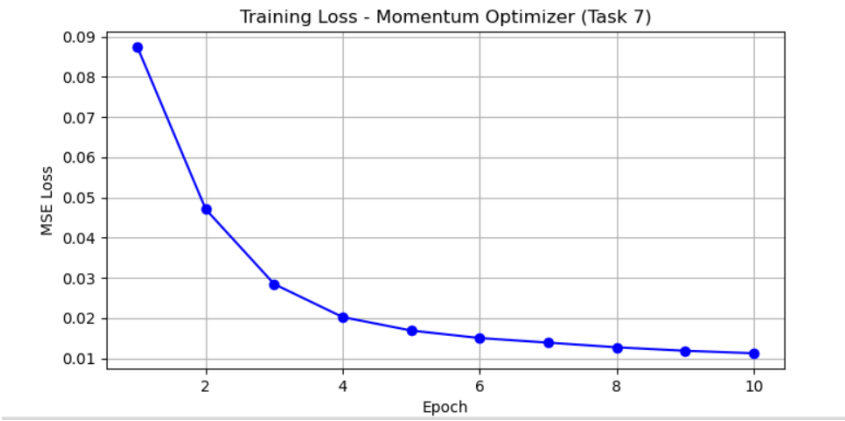

Figure 1: Loss Curve: Mean Squared Error vs. Epochs

5.2 Inference and Generalization

The final accuracy of **95.54%** demonstrates that the model generalizes well to unseen data. Figure 2 visualizes random test samples, comparing the ground truth with the network's prediction.

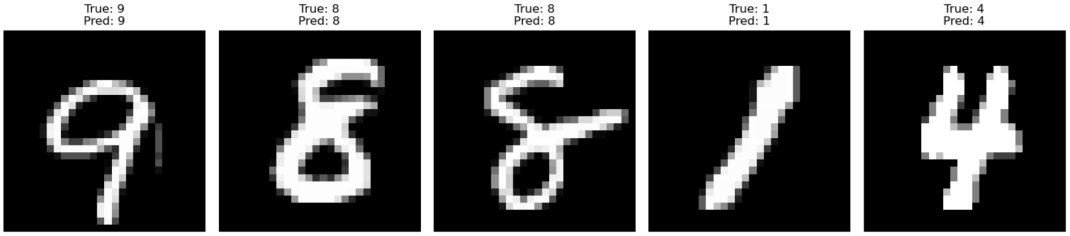

Figure 2: Visualizing Network Predictions on MNIST Test Set

6 Conclusion

The implementation of the 3-layer Neural Network successfully addresses the MNIST classification task. By manually deriving gradients and implementing momentum-based stochastic gradient descent, we achieved a high accuracy that competes with modern framework-based implementations. Future work could involve implementing the ReLU activation function and Xavier initialization to further enhance performance.